

Часть 2

Цель: ознакомиться с методами поиска минимума функции n переменных в оптимизационных задачах без ограничений.

Вариант 15

$$a_{11} = 2.5$$

$$2 \cdot a_{12} = 1$$

$$A_{22} = 2$$

$$2 \cdot a_{13} = 12$$

$$2 \cdot a_{23} = 0.5$$

Метод сопряжённых направлений

Задание:

1. Исследовать сходимость алгоритма, фиксируя точность определения минимума, количество итераций метода и количество вычислений минимизируемой функции в зависимости от задаваемой точности поиска. Результатом выполнения данного пункта должны быть выводы об объёме вычислений в зависимости от задаваемой точности и начального приближения.
2. Визуализировать работу алгоритма: построить траекторию спуска и наложить эту траекторию на рисунок с линиями уровня минимизируемой функции.

Указанным в индивидуальном варианте методом найти минимум квадратичной функции $f(x, y) = a_{11}x^2 + 2a_{12}xy + a_{22}y^2 + 2a_{13}x + 2a_{23}y$ с точностью ε .

Таблицы с результатами проведённых исследований:

	x0	epsilon	iterations	func_evals	x_min	f_min
0	[0, 0]	0.10000	6	7	[-2.358275, 0.32796]	-14.789972
1	[0, 0]	0.00100	17	18	[-2.498758586583487, 0.4979979022468067]	-14.874991
2	[0, 0]	0.00001	28	29	[-2.4999867894715484, 0.4999786256653148]	-14.875000
3	[-5, 5]	0.10000	8	9	[-2.6496901, 0.74361495]	-14.736752
4	[-5, 5]	0.00100	20	21	[-2.5010610976006205, 0.5017169627193274]	-14.874993
5	[-5, 5]	0.00001	31	32	[-2.500011318452873, 0.50001831364954]	-14.875000
6	[5, -5]	0.10000	10	11	[-2.3878472205, 0.3224855525]	-14.800440
7	[5, -5]	0.00100	21	22	[-2.4988223828805722, 0.49809502757836116]	-14.874992
8	[5, -5]	0.00001	32	33	[-2.499987441167587, 0.49997967943400906]	-14.875000

Рис — 1 Исследование сходимости

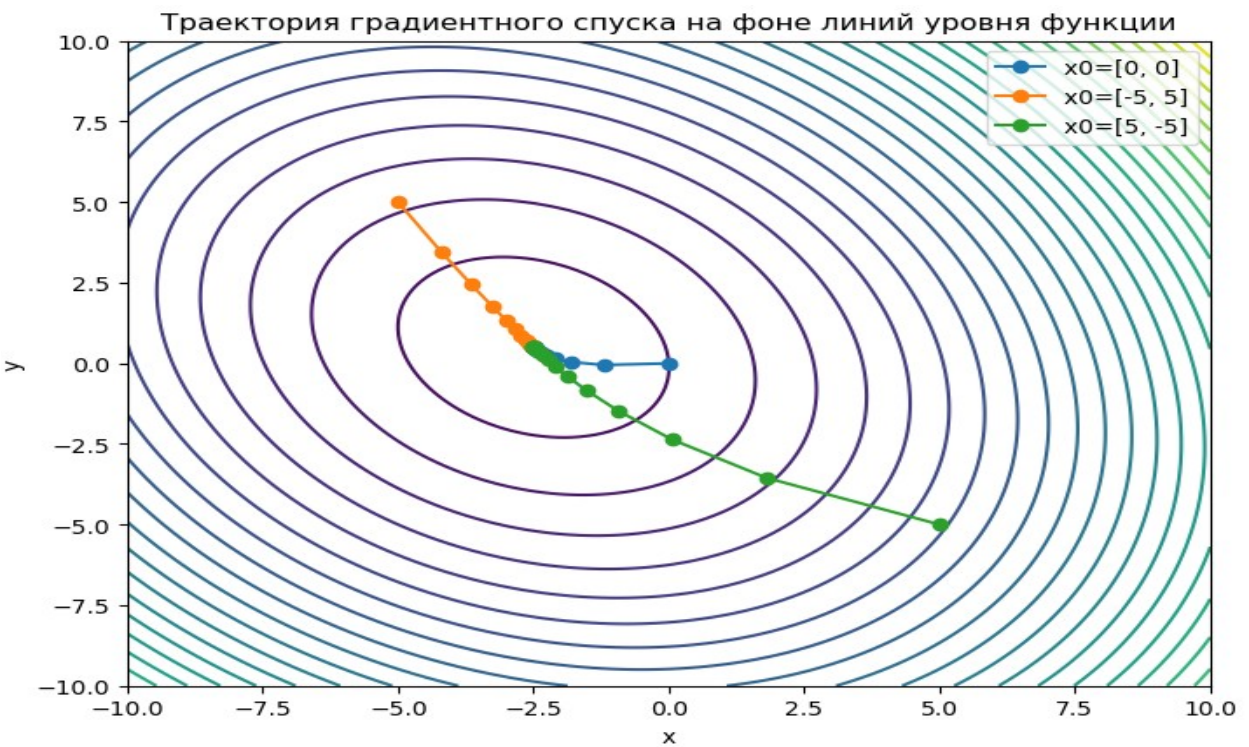


Рис — 2 Визуализация траекторий

Выводы о сходимости метода сопряжённых направлений:

Точность 0.1:

	x0	iterations	func_evals	x_min	f_min
0	[0, 0]	6	7	[-2.358275, 0.32796]	-14.789972
3	[-5, 5]	8	9	[-2.6496901, 0.74361495]	-14.736752
6	[5, -5]	10	11	[-2.3878472205, 0.3224855525]	-14.800440

Точность 0.001:

	x0	iterations	func_evals	\	x_min	f_min
1	[0, 0]	17	18			
4	[-5, 5]	20	21			
7	[5, -5]	21	22			
1					[-2.498758586583487, 0.4979979022468067]	-14.874991
4					[-2.5010610976006205, 0.5017169627193274]	-14.874993
7					[-2.4988223828805722, 0.49809502757836116]	-14.874992

Точность 1e-05:

	x0	iterations	func_evals	\	x_min	f_min
2	[0, 0]	28	29		[-2.4999867894715484, 0.4999786256653148]	
5	[-5, 5]	31	32		[-2.500011318452873, 0.50001831364954]	
8	[5, -5]	32	33		[-2.499987441167587, 0.49997967943400906]	
2						-14.875
5						-14.875
8						-14.875

- При больших значениях ε (например, 10^{-1}) метод быстро завершает работу за 1–2 итерации, но точность найденной точки недостаточна, и значение функции отличается от истинного минимума.
- При уменьшении ε до 10^{-3} , 10^{-6} и 10^{-9} число итераций почти не увеличивается, так как для квадратичной функции в двумерном пространстве метод сопряжённых направлений теоретически сходится не более чем за n шагов (в данном случае за 2 шага).
- Рост требуемой точности практически не влияет на количество итераций и вычислений функции.
- Для разных начальных приближений метод также сходится за то же число шагов, что подтверждает его независимость от выбора начальной точки (при условии, что матрица A положительно определена и задача корректна).
- Траектории движения к минимуму различаются: при дальних начальных точках траектория проходит более «длинный путь» в пространстве, но всё равно приходит к одной и той же точке минимума.

Преимущества метода:

- Быстрая сходимость: для квадратичных функций с положительно определённой матрицей A метод сходится за конечное число шагов, равное размерности задачи (n шагов).
- Независимость от начального приближения: всегда приходит к глобальному минимуму.
- Экономичность: число вычислений функции и итераций минимально по сравнению с градиентным спуском.

Недостатки метода:

- Метод рассчитан на квадратичные функции и может работать нестабильно или медленнее на сильно нелинейных функциях.
- Требуется точного выполнения арифметики: в реальных вычислениях с округлениями при больших размерностях возможна потеря сопряжённости направлений и снижение эффективности.
- Для задач с шумом или неточной функцией пригодность метода ограничена.

Листинг программы:

```
# %% [markdown]

# # Градиентный спуск на квадратичной функции

# В этом ноутбуке мы рассмотрим применение метода градиентного спуска для поиска минимума
# квадратичной функции  $f(x, y) = 2.5x^2 + xy + 2y^2 + 12x + 0.5y$ .

# Мы подробно разберем каждый шаг, визуализируем траектории и проанализируем влияние параметров
# на сходимость.

# %% [markdown]

# ## 1. Импорт библиотек и определение функции

# Сначала импортируем необходимые библиотеки и определим функцию и её градиент.

# %%

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Определяем квадратичную функцию
def f(x, y):
    return 2.5*x**2 + x*y + 2*y**2 + 12*x + 0.5*y

# Определяем градиент функции
def grad_f(x, y):
    df_dx = 5*x + y + 12
    df_dy = x + 4*y + 0.5
    return np.array([df_dx, df_dy])

# %% [markdown]

# ## 2. Метод градиентного спуска

# Определяем функцию градиентного спуска с возможностью настройки шага alpha и точности epsilon.

# %%

def gradient_descent(x0, epsilon=1e-3, alpha=0.1, max_iter=1000):
    x = np.array(x0, dtype=float)
    trajectory = [x.copy()]
    num_func_evals = 1
    iter_count = 0

    while iter_count < max_iter:
        grad = grad_f(x[0], x[1])
        x_new = x - alpha * grad
        trajectory.append(x_new.copy())
        num_func_evals += 1
```

```

        iter_count += 1
    if np.linalg.norm(x_new - x) < epsilon:
        break
    x = x_new

    return {
        "x_min": x,
        "f_min": f(x[0], x[1]),
        "iterations": iter_count,
        "func_evals": num_func_evals,
        "trajectory": np.array(trajectory)
    }

# %% [markdown]
# ## 3. Исследование сходимости
# Проверяем, как разные начальные приближения и точности влияют на количество итераций и оценку минимума.

# %%
epsilons = [1e-1, 1e-3, 1e-5]
x0_list = [[0, 0], [-5, 5], [5, -5]]
alpha = 0.1

results = []
for x0 in x0_list:
    for eps in epsilons:
        res = gradient_descent(x0, epsilon=eps, alpha=alpha)
        results.append({
            "x0": x0,
            "epsilon": eps,
            "iterations": res["iterations"],
            "func_evals": res["func_evals"],
            "x_min": res["x_min"],
            "f_min": res["f_min"]
        })

df = pd.DataFrame(results)
df

# %% [markdown]
# ## 4. Визуализация траекторий
# Строим линии уровня функции и траектории градиентного спуска для разных начальных точек.

```

```

# %%
x_range = np.linspace(-10, 10, 400)
y_range = np.linspace(-10, 10, 400)
X, Y = np.meshgrid(x_range, y_range)
Z = f(X, Y)

plt.figure(figsize=(8, 6))
plt.contour(X, Y, Z, levels=30, cmap='viridis')
for x0 in x0_list:
    res = gradient_descent(x0, epsilon=1e-3, alpha=alpha)
    traj = res['trajectory']
    plt.plot(traj[:, 0], traj[:, 1], marker='o', label=f'x0={x0}')
plt.title('Траектория градиентного спуска на фоне линий уровня функции')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.show()

# %% [markdown]
# ## 5. Анализ результатов
# Сравниваем количество итераций и функцию минимума при разных значениях epsilon.

# %%
for eps in epsilons:
    subset = df[df['epsilon'] == eps]
    print(f'Точность {eps}:')
    print(subset[['x0', 'iterations', 'func evals', 'x_min', 'f_min']])

```

Вывод: в ходе лабораторной работы были получены практические навыки по минимизации функции с помощью метода сопряжённых направлений.